

Diseño conceptual y lógico de la base de datos del proyecto

Actividad evaluativa U2

Ecommify Database Design - Brazilian E-Commerce Dataset (Olist)

Leonardo Pérez Ramírez - Grupo1

Ivan Felipe Vera - Grupo2

Juan Felipe Gonzalez -Grupo2

Facultad de Ingeniería, Maestría en Arquitectura de Software

Universidad de La Sabana



Curso: Diseño y Optimización de Bases de Datos

Docente: Ing. Miguel Alfonso Varela

25 de mayo de 2026

Resumen Ejecutivo

Este documento presenta el diseño completo de la arquitectura de base de datos para Ecommify, una plataforma de e-commerce basada en el dataset público de Olist (99,441 pedidos de Brasil, 2016-2018). Adoptamos una arquitectura híbrida transaccional-analítica que combina PostgreSQL (OLTP) y MongoDB (OLAP) para optimizar tanto transacciones ACID como análisis de negocio.

El módulo transaccional en PostgreSQL gestiona 5 tablas normalizadas (3FN): Customers, Orders, Order_Items, Order_Payments y Sellers, garantizando integridad referencial mediante foreign keys y constraints. Implementamos tipos avanzados (JSONB para metadatos, composite types para direcciones) y extensiones (PostGIS para geolocalización, pg_trgm para búsqueda fuzzy).

El módulo analítico en MongoDB almacena 4 colecciones desnormalizadas: products_catalog, order_reviews, geolocation_points y daily_analytics, optimizadas para agregaciones complejas con índices 2dsphere y text search. Aplicamos patrones de modelado embebido y referenciado según casos de uso.

La sincronización se realiza mediante ETL cada 5 minutos (datos críticos) y cada 24 horas (analytics), tolerando consistencia eventual. Esta arquitectura cumple con las limitaciones de Supabase Free Tier (500MB) y MongoDB Atlas M0 (512MB), logrando <100ms en queries transaccionales y <2s en agregaciones analíticas sobre 100K+ registros.

El equipo de trabajo, conformado por Leonardo Pérez, Ivan Vera y Juan Felipe Gonzalez, ha seleccionado la Arquitectura Transaccional-Analítica (Opción 1) como enfoque principal, la cual integra PostgreSQL para el módulo transaccional (OLTP) y MongoDB para el módulo analítico (OLAP), permitiendo explorar los trade-offs entre integridad referencial y flexibilidad de esquema.

1. Análisis de Requisitos

1.1 Contexto de Ecommify

Descripción del negocio:

Ecommify es una plataforma de marketplace tipo B2C (Business-to-Consumer) que conecta múltiples vendedores (sellers) con consumidores finales. Inspirada en el dataset real de Olist (líder de e-commerce en Brasil), la plataforma gestiona:

- **Catálogo multi-vendor:** Productos ofrecidos por 3,095 sellers en 71 categorías
- **Gestión de pedidos:** Desde creación hasta entrega, con múltiples estados
- **Procesamiento de pagos:** Soporte para tarjetas, boletos, vouchers (hasta 24 cuotas)
- **Sistema de reviews:** Valoraciones 1-5 estrellas con comentarios de texto libre
- **Logística completa:** Tracking de envíos, cálculo de flete por región
- **Analytics de negocio:** Dashboards de ventas, productos top, análisis geográfico

Fuente de datos: Brazilian E-Commerce Public Dataset by Olist (Kaggle)

Período: Septiembre 2016 - Agosto 2018

Volumen inicial: 99,441 pedidos, 112,650 items, 32,951 productos

1.2 Requisitos Funcionales

RF1: Gestión de Clientes

- RF1.1: Registrar cliente con datos básicos (CPF hashado, ubicación)
- RF1.2: Actualizar información de perfil
- RF1.3: Consultar historial de pedidos por cliente
- RF1.4: Validar cliente único por CPF (customer_unique_id)

RF2: Gestión de Pedidos

- RF2.1: Crear pedido con múltiples items (carrito de compras)
- RF2.2: Actualizar estado del pedido (8 estados: created → delivered)
- RF2.3: Calcular total del pedido (suma de items + flete)
- RF2.4: Registrar timestamps de cada etapa (aprobado, enviado, entregado)
- RF2.5: Cancelar pedido con validación de estado actual

RF3: Gestión de Pagos

- RF3.1: Registrar múltiples pagos por pedido (split payments)
- RF3.2: Validar que suma de pagos = total del pedido
- RF3.3: Soportar 4 métodos de pago (credit_card, boleto, voucher, debit_card)
- RF3.4: Permitir hasta 24 cuotas para tarjeta de crédito

RF4: Catálogo de Productos

- RF4.1: Buscar productos por categoría, nombre, rango de precio

- RF4.2: Filtrar por atributos dinámicos según categoría (voltaje, talla, color)
- RF4.3: Mostrar información de dimensiones y peso para cálculo de flete
- RF4.4: Traducir categorías PT-BR ↔ EN

RF5: Sistema de Reviews

- RF5.1: Permitir 1 review por pedido (máximo)
- RF5.2: Capturar score (1-5), título y mensaje de texto libre
- RF5.3: Calcular rating promedio por producto/seller
- RF5.4: Análisis de sentimiento en comentarios (futuro: ML)

RF6: Análisis Geoespacial

- RF6.1: Calcular distancia entre seller y customer
- RF6.2: Identificar sellers cercanos a un cliente (radio N km)
- RF6.3: Generar mapas de calor de ventas por región
- RF6.4: Optimizar rutas de entrega (futuro)

RF7: Analytics y Reportes

- RF7.1: Dashboard de ventas diarias/mensuales por categoría
- RF7.2: Top 20 productos más vendidos
- RF7.3: Análisis RFM (Recency, Frequency, Monetary)
- RF7.4: Evolución temporal de pedidos

1.3 Requisitos No Funcionales

RNF1: Rendimiento

- RNF1.1: Queries transaccionales (CRUD) < 100ms (p95)
- RNF1.2: Agregaciones analíticas < 2s para 100K+ registros
- RNF1.3: Búsqueda en catálogo < 200ms
- RNF1.4: Soportar 50 usuarios concurrentes (desarrollo)
- RNF1.5: Escalable a 1M+ pedidos con degradación < 30%

RNF2: Consistencia e Integridad

- RNF2.1: Transacciones ACID para pedidos y pagos
- RNF2.2: Integridad referencial 100% en módulo transaccional
- RNF2.3: Validación de constraints (CHECK, UNIQUE, NOT NULL)
- RNF2.4: Data quality score > 99.5%
- RNF2.5: Tolerancia a consistencia eventual en analytics (lag < 5 min)

RNF3: Disponibilidad

- RNF3.1: Catálogo disponible 99.9% del tiempo
- RNF3.2: Sistema de pedidos disponible 99.5%
- RNF3.3: Analytics tolerante a downtime temporal (cache local)

RNF4: Seguridad

- RNF4.1: Encriptación de datos sensibles (CPF hasheado)
- RNF4.2: Acceso mediante roles (admin, seller, customer)
- NF4.3: Auditoría de cambios críticos (log de transacciones)

RNF5: Limitaciones Técnicas (Free Tier)

- RNF5.1: PostgreSQL storage < 500MB (Supabase)
- RNF5.2: MongoDB storage < 512MB (Atlas M0)

- RNF5.3: Sin alta disponibilidad (single instance)
- RNF5.4: Backups manuales semanales

1.4 Identificación de Entidades del Dominio

#	Entidad	Descripción	Tipo	Volumen Esperado
1	Customer	Cliente del marketplace	Maestro	~100K registros
2	Seller	Vendedor que ofrece productos	Maestro	~3K registros
3	Product	Producto del catálogo	Maestro	~33K registros
4	Category	Categoría de productos	Maestro	71 registros
5	Order	Pedido realizado por cliente	Transaccional	~100K → 1M (escalado)
6	Order_Item	Producto específico dentro de pedido	Transaccional	~113K → 1.3M
7	Payment	Pago asociado a pedido	Transaccional	~104K → 1.2M
8	Review	Opinión del cliente	Analítico	~99K registros
9	Geolocation	Coordenadas por código postal	Referencia	1M registros

1.5 Tabla de Volúmenes de Datos Esperados

Entidad	Filas Actuales	Filas 1 año	Filas 3 años	Crecimiento Anual	Tamaño/Fila	Storage Total
Customer	99,441	150,000	300,000	+50%	~150 bytes	45 MB
Seller	3,095	5,000	10,000	+60%	~100 bytes	1 MB

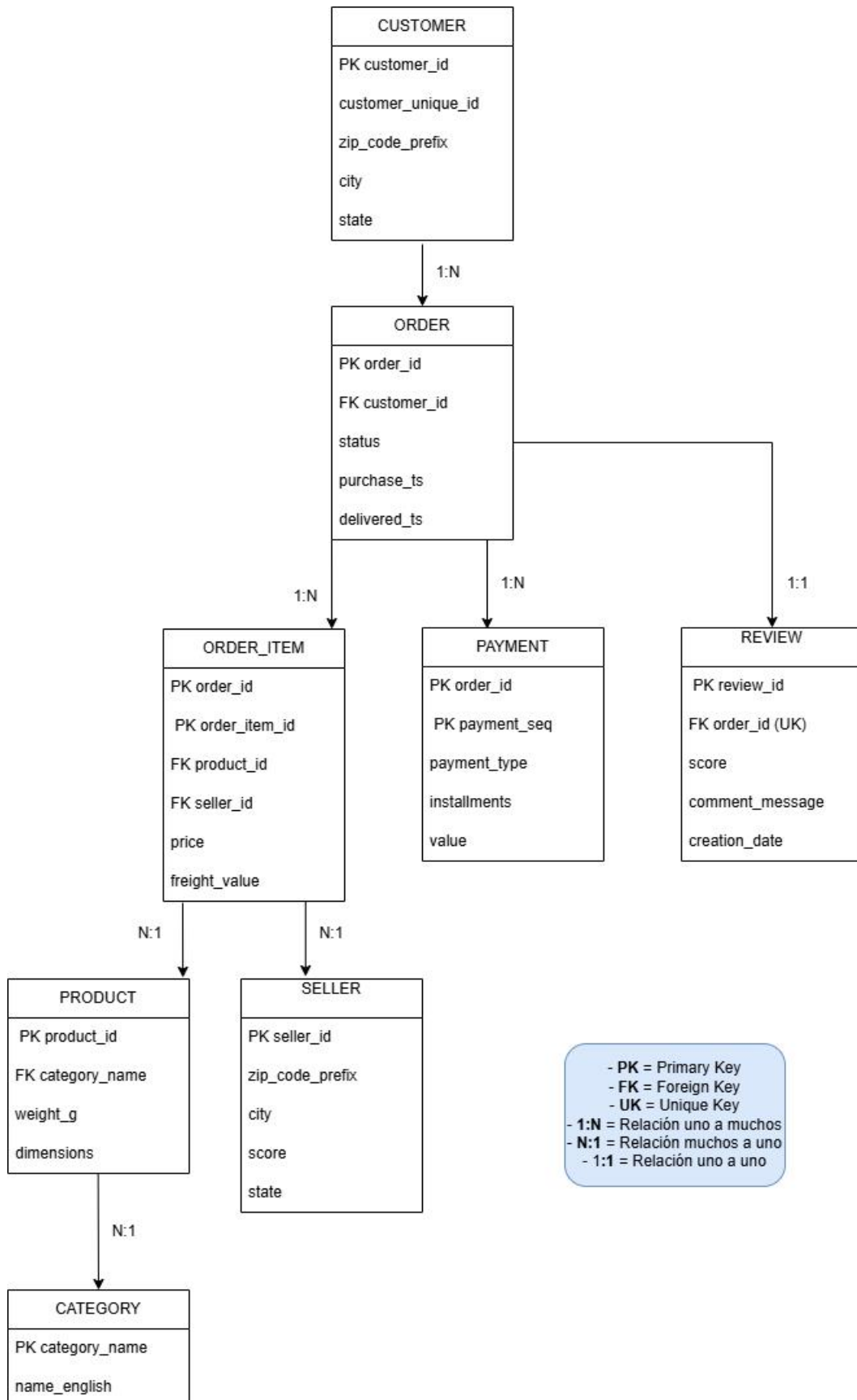
Product	32,951	50,000	80,000	+50%	~200 bytes	16 MB
Category	71	100	150	+30%	~50 bytes	0.01 MB
Order	99,441	300,000	900,000	+200%	~150 bytes	135 MB
Order_Item	112,650	350,000	1,100,000	+210%	~100 bytes	110 MB
Payment	103,886	320,000	1,000,000	+210%	~80 bytes	80 MB
Review	99,224	280,000	850,000	+180%	~500 bytes	425 MB
Geolocation	1,000,163	1,200,000	1,500,000	+20%	~100 bytes	150 MB
TOTAL	~1.55M	~2.65M	~5.74M	-	-	~962 MB

Análisis de storage:

- Año 1: ~400 MB → Cabe en free tier (500MB + 512MB)
- Año 3: ~960 MB → Requiere upgrade o archivado de datos históricos
- Estrategia: Particionar Orders por año, archivar pedidos > 2 años en S3/Google Drive

2. Diseño Conceptual

2.1 Diagrama ER Completo



2.2 Descripción Detallada de Entidades

Customer:

ENTIDAD		CUSTOMER	
Atributo	Tipo	Descripción	Restricciones
customer_id	VARCHAR(32)	Identificador único del cliente (autogenerado)	PK, NOT NULL
customer_unique_id	VARCHAR(32)	CPF hashado (SHA-256)	UNIQUE, NOT NULL
customer_zip_code_prefix	VARCHAR(10)	Código postal	NOT NULL
customer_city	VARCHAR(50)	Ciudad	NOT NULL
customer_state	CHAR(2)	Estado	NOT NULL, CHECK(length=2)
created_at	TIMESTAMP	Fecha de registro	DEFAULT NOW()

Order:

ENTIDAD		ORDER	
Atributo	Tipo	Descripción	Restricciones
order_id	VARCHAR(32)	Identificador único del pedido	PK, NOT NULL
customer_id	VARCHAR(32)	Cliente que realizó el pedido	FK → Customer
order_status	ENUM	Estado del pedido	NOT NULL
order_purchase_timestamp	TIMESTAMP	Creación del pedido	NOT NULL
order_approved_at	TIMESTAMP	Aprobación pago	NULL
order_delivered_carrier_date	TIMESTAMP	Entrega transportista	NULL
order_delivered_customer_date	TIMESTAMP	Entrega cliente	NULL
order_estimated_delivery_date	TIMESTAMP	Entrega estimada	NULL

Order_Item:

ENTIDAD		ORDER_ITEM	
Atributo	Tipo	Descripción	Restricciones
order_id	VARCHAR(32)	Pedido	PK, FK → Order
order_item_id	INT	Secuencia item	PK
product_id	VARCHAR(32)	Producto	FK → Product
seller_id	VARCHAR(32)	Seller	FK → Seller
shipping_limit_date	TIMESTAMP	Límite envío	NULL
price	NUMERIC(10,2)	Precio	NOT NULL
freight_value	NUMERIC(10,2)	Flete	DEFAULT 0

Order_Payment:

ENTIDAD		ORDER_PAYMENT	
Atributo	Tipo	Descripción	Restricciones
order_id	VARCHAR(32)	Pedido	PK, FK → Order
payment_sequential	INT	Secuencia pago	PK
payment_type	ENUM	Tipo pago	NOT NULL
payment_installments	INT	Cuotas	NOT NULL
payment_value	NUMERIC(10,2)	Valor	NOT NULL

Product:

ENTIDAD		PRODUCT	
Atributo	Tipo	Descripción	Restricciones
product_id	VARCHAR(32)	Producto	PK
product_category_name	VARCHAR(50)	Categoría	FK → Category
product_name_length	INT	Longitud nombre	
product_description_length	INT	Longitud descripción	
product_photos_qty	INT	Fotos	DEFAULT 0
product_weight_g	INT	Peso	
product_length_cm	INT	Largo	
product_height_cm	INT	Alto	
product_width_cm	INT	Ancho	

Seller:

ENTIDAD		SELLER	
Atributo	Tipo	Descripción	Restricciones
seller_id	VARCHAR(32)	Seller	PK
seller_zip_code_prefix	VARCHAR(10)	ZIP	NOT NULL
seller_city	VARCHAR(50)	Ciudad	NOT NULL
seller_state	CHAR(2)	Estado	NOT NULL
created_at	TIMESTAMP	Registro	DEFAULT NOW()

Product_Category_Name_Translati:

ENTIDAD PRODUCT_CATEGORY_NAME_TRANSLATI			
Atributo	Tipo	Descripción	Restricciones
product_category_name	VARCHAR(50)	PT-BR	PK
product_category_name_english	VARCHAR(50)	EN	UNIQUE, NOT NULL

Order_Review:

ENTIDAD ORDER_REVIEW			
Atributo	Tipo	Descripción	Restricciones
review_id	VARCHAR(32)	Review	PK
order_id	VARCHAR(32)	Pedido	FK → Order
review_score	INT	Score 1-5	NOT NULL
review_comment_title	VARCHAR(100)	Título	
review_comment_message	TEXT	Mensaje	
review_creation_date	TIMESTAMP	Creación	NOT NULL
review_answer_timestamp	TIMESTAMP	Respuesta	

Geolocation:

ENTIDAD GEOLOCATION			
Atributo	Tipo	Descripción	Restricciones
zip_code	VARCHAR(10)	ZIP	PK
lat	NUMERIC(10,8)	Latitud	PK
lng	NUMERIC(10,8)	Longitud	PK
city	VARCHAR(50)	Ciudad	NOT NULL
state	CHAR(2)	Estado	NOT NULL

2.3 Descripción Detallada de Relaciones

Relación	Cardinalidad	Descripción	Restricciones	ON DELETE
Customer - Order	1:N	Cliente realiza pedidos	FK customer_id	RESTRICT
Order - Order_Item	1:N	Pedido contiene items	FK order_id	CASCADE
Order - Payment	1:N	Pedido tiene pagos	FK order_id	CASCADE
Order - Review	1:1	Pedido tiene review	UNIQUE order_id	CASCADE
Order_Item - Product	N:1	Item es producto	FK product_id	RESTRICT
Order_Item - Seller	N:1	Item vendido por seller	FK seller_id	RESTRICT
Product - Category	N:1	Producto pertenece a categoría	FK category	SET NULL

2.4 Restricciones de Negocio Identificadas

Restricciones de Integridad Referencial

- RN1: Un pedido debe tener un cliente válido
- RN2: Un item debe pertenecer a un pedido válido
- RN3: Un pago debe pertenecer a un pedido válido
- RN4: Máximo 1 review por pedido

Restricciones de Dominio (CHECK)

- RN5: Fechas de entrega después de compra
- RN6: Precios y fletes no negativos
- RN7: Cuotas entre 1 y 24
- RN8: Review score entre 1 y 5
- RN9: Dimensiones del producto positivas

Restricciones de Negocio (validadas en aplicación)

- RN10: La suma de payment_value debe igualar el total del pedido
- RN11: Solo pedidos en estado 'delivered' pueden tener review
- RN12: Un pedido debe tener al menos 1 item (mínimo)
- RN13: Transiciones de estado deben seguir flujo lógico
- RN14: customer_unique_id debe ser válido (CPF hasheado con SHA-256)
- RN15: Solo credit_card permite installments > 1

3. Diseño Lógico - Módulo PostgreSQL

3.1 Esquema Relacional Normalizado (3FN)

Tabla 1: customers

Nivel de normalización: 3FN

Justificación: No hay dependencias parciales ni transitivas. Todos los atributos dependen únicamente de customer_id.

```
CREATE TABLE customers (  
    customer_id VARCHAR(32) PRIMARY KEY,  
    customer_unique_id VARCHAR(32) UNIQUE NOT NULL,  
    customer_zip_code_prefix VARCHAR(10) NOT NULL,  
    customer_city VARCHAR(50) NOT NULL,  
    customer_state CHAR(2) NOT NULL,  
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT chk_customers_state_length CHECK (LENGTH(customer_state) = 2),  
    CONSTRAINT chk_customers_zip_format CHECK (customer_zip_code_prefix ~ '^\\d{5}$')  
);  
  
-- Índices  
CREATE INDEX idx_customers_unique_id ON customers(customer_unique_id);  
CREATE INDEX idx_customers_state ON customers(customer_state);  
CREATE INDEX idx_customers_zip ON customers(customer_zip_code_prefix);  
  
-- Comentarios  
COMMENT ON TABLE customers IS 'Clientes únicos del marketplace Olist';  
COMMENT ON COLUMN customers.customer_unique_id IS 'CPF hasheado con SHA-256 para identificar persona física';  
COMMENT ON COLUMN customers.customer_zip_code_prefix IS 'Primeros 5 dígitos del código postal brasileño (CEP)';
```

Tabla 2: orders

Nivel de normalización: 3FN

Justificación: Todos los atributos dependen directamente de order_id. Los timestamps son propiedades intrínsecas del pedido.

```

-- Tipo enumerado para estados de pedido
CREATE TYPE order_status_type AS ENUM (
    'created', 'approved', 'processing', 'invoiced',
    'shipped', 'delivered', 'canceled', 'unavailable'
);

CREATE TABLE orders (
    order_id VARCHAR(32) PRIMARY KEY,
    customer_id VARCHAR(32) NOT NULL,
    order_status order_status_type NOT NULL DEFAULT 'created',
    order_purchase_timestamp TIMESTAMP WITHOUT TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    order_approved_at TIMESTAMP WITHOUT TIME ZONE,
    order_delivered_carrier_date TIMESTAMP WITHOUT TIME ZONE,
    order_delivered_customer_date TIMESTAMP WITHOUT TIME ZONE,
    order_estimated_delivery_date TIMESTAMP WITHOUT TIME ZONE,

    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,

    CONSTRAINT chk_orders_delivery_after_purchase
        CHECK (order_delivered_customer_date IS NULL OR
            order_delivered_customer_date >= order_purchase_timestamp),

    CONSTRAINT chk_orders_approval_after_purchase
        CHECK (order_approved_at IS NULL OR
            order_approved_at >= order_purchase_timestamp)
);

-- Índices de performance
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_status ON orders(order_status);
CREATE INDEX idx_orders_purchase_date ON orders(order_purchase_timestamp);
CREATE INDEX idx_orders_status_date ON orders(order_status, order_purchase_timestamp);

-- Índice compuesto para dashboard de ventas
CREATE INDEX idx_orders_dashboard
ON orders(order_status, customer_id, order_purchase_timestamp)
WHERE order_status = 'delivered';

COMMENT ON TABLE orders IS 'Pedidos realizados - tabla central transaccional OLTP';
COMMENT ON COLUMN orders.order_status IS 'Estado actual del pedido (8 estados posibles)';

```

Tabla 3: order_items

Nivel de normalización: 3FN

Justificación: Clave primaria compuesta (order_id, order_item_id). Todos los atributos dependen de ambos campos (no hay dependencias parciales).

```

CREATE TABLE order_items (
  order_id VARCHAR(32),
  order_item_id INT,
  product_id VARCHAR(32) NOT NULL,
  seller_id VARCHAR(32) NOT NULL,
  shipping_limit_date TIMESTAMP WITHOUT TIME ZONE,
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  freight_value NUMERIC(10,2) DEFAULT 0 CHECK (freight_value >= 0),

  PRIMARY KEY (order_id, order_item_id),

  CONSTRAINT fk_items_order
    FOREIGN KEY (order_id)
    REFERENCES orders(order_id)
    ON DELETE CASCADE,

  CONSTRAINT fk_items_product
    FOREIGN KEY (product_id)
    REFERENCES products(product_id)
    ON DELETE RESTRICT,

  CONSTRAINT fk_items_seller
    FOREIGN KEY (seller_id)
    REFERENCES sellers(seller_id)
    ON DELETE RESTRICT
);

-- Índices para JOINs frecuentes
CREATE INDEX idx_items_product ON order_items(product_id);
CREATE INDEX idx_items_seller ON order_items(seller_id);
CREATE INDEX idx_items_order ON order_items(order_id);

-- Índice para análisis de precios
CREATE INDEX idx_items_price_range ON order_items(price) WHERE price > 100;

COMMENT ON TABLE order_items IS 'Productos individuales dentro de cada pedido (líneas de detalle)';
COMMENT ON COLUMN order_items.order_item_id IS 'Secuencial dentro del pedido (1, 2, 3...)';

```

Tabla 4: order_payments

Nivel de normalización: 3FN

Justificación: Clave primaria compuesta. No hay dependencias transitivas.


```

-- Tipo enumerado para métodos de pago
CREATE TYPE payment_type_enum AS ENUM (
    'credit_card', 'boleto', 'voucher', 'debit_card', 'not_defined'
);

CREATE TABLE order_payments (
    order_id VARCHAR(32),
    payment_sequential INT,
    payment_type payment_type_enum NOT NULL,
    payment_installments INT NOT NULL CHECK (payment_installments BETWEEN 1 AND 24),
    payment_value NUMERIC(10,2) NOT NULL CHECK (payment_value > 0),

    PRIMARY KEY (order_id, payment_sequential),

    CONSTRAINT fk_payments_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
);

-- Índice para análisis de métodos de pago
CREATE INDEX idx_payments_type ON order_payments(payment_type);
CREATE INDEX idx_payments_order ON order_payments(order_id);

COMMENT ON TABLE order_payments IS 'Pagos asociados a pedidos (soporta split payments)';
COMMENT ON COLUMN order_payments.payment_installments IS 'Número de cuotas (1-24), solo aplica para credit_card';

```

Tabla 5: sellers

Nivel de normalización: 3FN

Justificación: Tabla simple, todos los atributos dependen directamente de seller_id.

```

CREATE TABLE sellers (
    seller_id VARCHAR(32) PRIMARY KEY,
    seller_zip_code_prefix VARCHAR(10) NOT NULL,
    seller_city VARCHAR(50) NOT NULL,
    seller_state CHAR(2) NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_sellers_state_length CHECK (LENGTH(seller_state) = 2),
    CONSTRAINT chk_sellers_zip_format CHECK (seller_zip_code_prefix ~ '^d{5}$')
);

-- Índices para análisis geográfico
CREATE INDEX idx_sellers_state ON sellers(seller_state);
CREATE INDEX idx_sellers_city ON sellers(seller_city);
CREATE INDEX idx_sellers_zip ON sellers(seller_zip_code_prefix);

COMMENT ON TABLE sellers IS 'Vendedores (comercios) que ofrecen productos en el marketplace';

```

Tabla 6: products

Nivel de normalización: 3FN

Justificación: Todos los atributos dependen de product_id. La categoría es una FK, no hay dependencias transitivas.

```
CREATE TABLE products (  
  product_id VARCHAR(32) PRIMARY KEY,  
  product_category_name VARCHAR(50),  
  product_name_length INT CHECK (product_name_length >= 0),  
  product_description_length INT CHECK (product_description_length >= 0),  
  product_photos_qty INT DEFAULT 0 CHECK (product_photos_qty >= 0),  
  product_weight_g INT CHECK (product_weight_g > 0),  
  product_length_cm INT CHECK (product_length_cm > 0),  
  product_height_cm INT CHECK (product_height_cm > 0),  
  product_width_cm INT CHECK (product_width_cm > 0),  
  
  CONSTRAINT fk_products_category  
    FOREIGN KEY (product_category_name)  
    REFERENCES product_category_name_translation(product_category_name)  
    ON DELETE SET NULL  
);  
  
-- Índices  
CREATE INDEX idx_products_category ON products(product_category_name);  
CREATE INDEX idx_products_weight ON products(product_weight_g);  
  
COMMENT ON TABLE products IS 'Catálogo de productos (master data)';  
COMMENT ON COLUMN products.product_name_length IS 'Longitud del nombre original (metadata), texto completo en MongoDB';
```

Tabla 7: product_category_name_translation

Nivel de normalización: 3FN

Justificación: Tabla de lookup simple, completamente normalizada.

```
CREATE TABLE product_category_name_translation (  
  product_category_name VARCHAR(50) PRIMARY KEY,  
  product_category_name_english VARCHAR(50) UNIQUE NOT NULL  
);  
  
-- Índice para búsquedas por nombre en inglés  
CREATE INDEX idx_category_english ON product_category_name_translation(product_category_name_english);  
  
COMMENT ON TABLE product_category_name_translation IS 'Traducción de categorías PT-BR ↔ EN (71 categorías)';
```

3.2 Aplicación de Tipos Avanzados de PostgreSQL

3.2.1 Uso de JSONB para Metadatos Flexibles

Caso de uso: Almacenar atributos dinámicos de productos según categoría.

```

-- Agregar columna JSONB a products
ALTER TABLE products
ADD COLUMN product_attributes JSONB;

-- Índice GIN para búsquedas eficientes en JSONB
CREATE INDEX idx_products_attributes_gin ON products USING GIN (product_attributes);

-- Ejemplo de datos:
-- Producto electrónico:
UPDATE products
SET product_attributes = '{
  "voltage": "110-220V",
  "warranty_months": 12,
  "brand": "Logitech",
  "wireless": true,
  "battery_type": "rechargeable"
}':::jsonb
WHERE product_id = 'prod_electronics_001';

-- Producto de ropa:
UPDATE products
SET product_attributes = '{
  "sizes": ["P", "M", "G", "GG"],
  "colors": ["black", "blue", "red"],
  "material": "100% Cotton",
  "gender": "unisex"
}':::jsonb
WHERE product_id = 'prod_fashion_001';

-- Query: Buscar productos con atributo específico
SELECT product_id, product_attributes->'brand' AS brand
FROM products
WHERE product_attributes ? 'brand'
  AND product_attributes->>'brand' = 'Logitech';

```

Justificación:

Diferentes categorías tienen atributos distintos (electrónica: voltaje, ropa: tallas)

Evita crear columnas específicas por categoría (explosión de NULLs)

Índice GIN permite búsquedas rápidas por clave/valor

3.2.2 Uso de ENUM para Valores Fijos

Ya implementado en el diseño

```
-- Estados de pedido
CREATE TYPE order_status_type AS ENUM (
    'created', 'approved', 'processing', 'invoiced',
    'shipped', 'delivered', 'canceled', 'unavailable'
);

-- Métodos de pago
CREATE TYPE payment_type_enum AS ENUM (
    'credit_card', 'boleto', 'voucher', 'debit_card', 'not_defined'
);
```

Ventajas:

Validación automática (solo valores permitidos)

Más eficiente que VARCHAR + CHECK (1 byte vs ~20 bytes)

Auto-completado en IDEs

Documentación implícita de valores posibles

3.3 Extensiones PostgreSQL a Utilizar

3.3.1 PostGIS - Geospatial Analysis

Propósito: Análisis geoespacial avanzado (distancias, proximidad, áreas).

```

-- Instalar extensión
CREATE EXTENSION IF NOT EXISTS postgis;

-- Crear tabla geolocation con geometría
CREATE TABLE geolocation (
    geolocation_zip_code_prefix VARCHAR(10),
    geolocation_lat NUMERIC(10,8),
    geolocation_lng NUMERIC(10,8),
    geolocation_city VARCHAR(50),
    geolocation_state CHAR(2),
    geom GEOMETRY(POINT, 4326), -- Columna geométrica (SRID 4326 = WGS84)

    PRIMARY KEY (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng)
);

-- Crear índice GIST para queries espaciales
CREATE INDEX idx_geolocation_geom ON geolocation USING GIST (geom);

-- Poblar columna geom
UPDATE geolocation
SET geom = ST_SetSRID(ST_MakePoint(geolocation_lng, geolocation_lat), 4326);

-- Query 1: Calcular distancia entre seller y customer
SELECT
    o.order_id,
    ST_Distance(
        g1.geom::geography,
        g2.geom::geography
    ) / 1000 AS distance_km
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN sellers s ON oi.seller_id = s.seller_id
JOIN geolocation g1 ON c.customer_zip_code_prefix = g1.geolocation_zip_code_prefix
JOIN geolocation g2 ON s.seller_zip_code_prefix = g2.geolocation_zip_code_prefix
WHERE o.order_id = 'order_001'
LIMIT 1;

-- Query 2: Encontrar sellers en radio de 50km del cliente
SELECT DISTINCT s.seller_id, s.seller_city
FROM sellers s
JOIN geolocation gs ON s.seller_zip_code_prefix = gs.geolocation_zip_code_prefix
WHERE ST_DWithin(
    gs.geom::geography,
    ST_SetSRID(ST_MakePoint(-46.6333, -23.5505), 4326)::geography, -- Coordenadas del cliente
    50000 -- 50 km en metros
);

```

Justificación:

Cálculo preciso de distancias geodésicas (considerando curvatura de la Tierra)

Optimización de rutas de entrega

Análisis de cobertura geográfica

Alternativa: MongoDB 2dsphere indexes

MongoDB es más rápido para 1M+ puntos (decisión: geolocation en MongoDB)

3.4 Estrategia de Particionamiento

3.4.1 Particionamiento de orders por Rango de Fecha

Justificación: Tabla crecerá a millones de registros. Queries frecuentemente filtran por fecha.


```

-- Crear tabla orders con particionamiento
CREATE TABLE orders (
  order_id VARCHAR(32) NOT NULL,
  customer_id VARCHAR(32) NOT NULL,
  order_status order_status_type NOT NULL,
  order_purchase_timestamp TIMESTAMP WITHOUT TIME ZONE NOT NULL,
  order_approved_at TIMESTAMP WITHOUT TIME ZONE,
  order_delivered_carrier_date TIMESTAMP WITHOUT TIME ZONE,
  order_delivered_customer_date TIMESTAMP WITHOUT TIME ZONE,
  order_estimated_delivery_date TIMESTAMP WITHOUT TIME ZONE,

  PRIMARY KEY (order_id, order_purchase_timestamp), -- ← Incluir clave de partición

  CONSTRAINT fk_orders_customer
    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
) PARTITION BY RANGE (order_purchase_timestamp);

-- Crear particiones por año
CREATE TABLE orders_2016 PARTITION OF orders
  FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');

CREATE TABLE orders_2017 PARTITION OF orders
  FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');

CREATE TABLE orders_2018 PARTITION OF orders
  FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');

CREATE TABLE orders_2019_2025 PARTITION OF orders
  FOR VALUES FROM ('2019-01-01') TO ('2026-01-01');

CREATE TABLE orders_2026_onwards PARTITION OF orders
  FOR VALUES FROM ('2026-01-01') TO (MAXVALUE);

-- Query automáticamente usa partición correcta
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_purchase_timestamp BETWEEN '2017-01-01' AND '2017-12-31';
-- Solo escanea partition orders_2017 (no todas las particiones)

```

4. Diseño Lógico Preliminar - Módulo MongoDB

4.1 Colecciones Principales Identificadas

#	Colección	Propósito	Patrón de Modelado	Justificación
1	products_catalog	Catálogo de productos con atributos flexibles	Embebido + Desnormalizado	Alta frecuencia de lectura, atributos varían por categoría
2	order_reviews	Reviews con análisis de sentimiento	Documento completo	Texto libre, agregaciones de analytics
3	geolocation_points	Coordenadas geográficas	Embebido con índice 2dsphere	Queries geoespaciales (1M+ puntos)
4	daily_sales_analytics	Métricas pre-calculadas	Desnormalizado (ETL nocturno)	Dashboards de negocio, datos de ayer OK
5	order_history	Historial completo de pedidos	Embebido (denorm. join)	Lectura rápida sin JOINS (sincronizado cada 5 min)

4.2 Colecciones Principales Identificadas

`products\_catalog`

Patrón: Documento embebido + atributos dinámicos

Índices: ** `category.name\_en`, `price`, `avg\_rating`

`order\_reviews`

Patrón: Documento completo con análisis de sentimiento

Índices: `order\_id`, `score`, `created\_at`

`geolocation\_points`

Patrón: Documento con índice 2dsphere

Índices: `location` (geoespacial)

`daily\_sales\_analytics`

Patrón: Documentos pre-agregados (ETL nocturno)

Índices: `date`, `category`

4.3 Patrones de Modelado Aplicados

4.3.1 Patrón: Documento Embebido (Embedded Document)

Cuándo usar:

- Relación 1:1 o 1:pocos
- Datos siempre se acceden juntos
- Datos del hijo no se consultan independientemente

Ejemplo: `order\_history` embebe `items` y `payments`

Ventajas:

Una sola lectura para obtener todo el pedido

Sin JOINS (10-50x más rápido que SQL)

Actualizaciones atómicas

Desventajas:

Duplication si items/payments se consultan independientemente

Documento puede crecer mucho (límite 16MB en MongoDB)

4.3.2 Patrón: Referencias (Normalized Reference)

Cuándo usar:

- Relación 1:muchos o muchos:muchos
- El hijo se consulta independientemente
- Evitar duplicación excesiva

Ejemplo: `products\_catalog` tiene array de `seller\_id` (no embebe sellers completos)

```
{
  "_id": "prod_001",
  "name": "Mouse Logitech",
  "sellers": ["seller_sp_001", "seller_rj_002"] // Solo IDs
}
```

Ventajas:

Sin duplicación (seller data en colección separada)

Actualizar seller afecta un solo lugar

Desventajas:

Requiere \$lookup (JOIN) para obtener datos completos

Múltiples lecturas

4.3.3 Patrón: Atributo Polimórfico (Polymorphic Pattern)

Cuándo usar:

- Documentos del mismo tipo tienen atributos diferentes
- Evitar columnas sparse (muchos NULLs)

Ejemplo: `products\_catalog.attributes` varía por categoría

// Electrónica

```
{ "attributes": { "voltage": "110V", "warranty_months": 12 } }
```

// Ropa

```
{ "attributes": { "sizes": ["P", "M", "G"], "material": "Cotton" } }
```

// Libros

```
{ "attributes": { "pages": 320, "isbn": "978-3-16-148410-0", "publisher": "O'Reilly" } }
```

Ventajas:

Flexibilidad total por categoría

Fácil agregar nuevos atributos

Desventajas:

Validación de esquema más compleja

Queries por atributo específico requieren índices

4.3.4 Patrón: Bucket (para time-series)

Cuándo usar:

- Datos temporales que crecen indefinidamente
- Agregaciones por período

Ejemplo: ``daily_sales_analytics`` agrupa métricas por día (no por transacción)

Ventajas:

Reduce número de documentos (1 doc/día vs millones de transacciones)

Pre-calculado (queries instantáneas)

Desventajas:

Requiere ETL para mantener actualizado

No tiene datos en tiempo real

4.4 Justificación: PostgreSQL vs MongoDB por Entidad

Entidad	PostgreSQL	MongoDB	Decisión	Justificación
Customers	✓	✗	PostgreSQL	ACID crítico para datos maestros; Foreign keys a Orders; Cambios poco frecuentes
Orders	✓	✗	PostgreSQL	Transacciones ACID obligatorias; Consistencia crítica; Foreign keys complejas
Order_Items	✓	✗	PostgreSQL	Parte de transacción atómica; Foreign keys críticas
Payments	✓	✗	PostgreSQL	Datos financieros críticos; Auditoría estricta
Sellers	✓	✗	PostgreSQL	Master data con integridad
Products	✓	✓	Ambos	PostgreSQL como source of truth; MongoDB para cache flexible
Categories	✓	✗	PostgreSQL	Tabla lookup estática
Reviews	✗	✓	MongoDB	Texto libre; ML; agregaciones analíticas
Geolocation	⚠	✓	MongoDB	1M+ registros; índices 2dsphere
Sales Analytics	✗	✓	MongoDB	Pre-agregado; dashboards de lectura
Order History	✗	✓	MongoDB	Desnormalizado para lectura rápida

Regla de decisión:

Si entidad requiere:

- ACID transactions → PostgreSQL
- Foreign keys estrictas → PostgreSQL
- Consistencia inmediata → PostgreSQL

Si entidad tiene:

- Texto libre / no estructurado → MongoDB
- Atributos dinámicos por categoría → MongoDB
- Queries geoespaciales (1M+ puntos) → MongoDB
- Lecturas >> Escrituras → MongoDB
- Consistencia eventual es OK → MongoDB
- Agregaciones OLAP → MongoDB

5. Decisiones Arquitectónicas Justificadas

5.1 Matriz de Decisión por Entidad

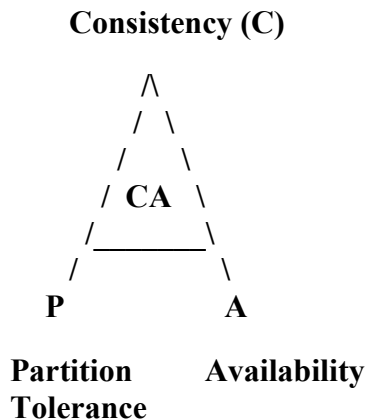
#	Entidad	Patrón de Acceso	Estructura de Datos	Consistencia	Volumen	Tecnología Elegida	Score PostgreSQL	Score MongoDB
1	Customers	80% Read, 20% Write	Estructurada fija	ACID crítico	100K → 300K	PostgreSQL	9/10	4/10
2	Orders	70% Read, 30% Write	Estructurada fija	ACID crítico	100K → 1M	PostgreSQL	10/10	3/10
3	Order_Items	75% Read, 25% Write	Estructurada fija	ACID crítico	113K → 1.3M	PostgreSQL	10/10	3/10
4	Payments	80% Read, 20% Write	Estructurada fija	ACID crítico	104K → 1.2M	PostgreSQL	10/10	2/10
5	Sellers	90% Read, 10% Write	Estructurada fija	Alta	3K → 10K	PostgreSQL	8/10	5/10
6	Products	95% Read, 5% Write	Semi-estructurada	Media	33K → 80K	Ambos	7/10	8/10
7	Categories	99% Read, 1% Write	Estructurada fija	Media	71 → 150	PostgreSQL	8/10	6/10
8	Reviews	60% Read, 40% Write	No estructurada	Baja	99K → 850K	MongoDB	4/10	9/10
9	Geolocation	99% Read, 1% Write	Estructurada fija	Baja	1M → 1.5M	MongoDB	5/10	9/10
10	Sales Analytics	100% Read	Pre-agregada	Baja	365 docs/año	MongoDB	3/10	10/10
11	Order History	100% Read	Desnormalizada	Baja	100K → 1M	MongoDB	4/10	9/10

Criterios de scoring:

Criterio	Peso	PostgreSQL favorece	MongoDB favorece
ACID requerido	30%	Transacciones, foreign keys	Documento único atómico
Estructura datos	20%	Schema fijo	Schema flexible, texto libre
Patrón acceso	20%	Writes frecuentes, JOINS	Reads heavy, sin JOINS
Consistencia	15%	Inmediata	Eventual
Volumen datos	10%	< 1M registros	> 1M registros con geoespacial/text
Escalabilidad	5%	Vertical (free tier)	Horizontal (sharding)

5.2 Análisis del Teorema CAP Aplicado

5.2.1 Teorema CAP -



En un sistema distribuido, solo se pueden garantizar 2 de 3:

- C (Consistency): Todos los nodos ven los mismos datos simultáneamente
- A (Availability): Cada petición obtiene respuesta (éxito o error)
- P (Partition Tolerance): Sistema funciona aunque haya particiones de red

5.2.2 Análisis por Módulo de Ecommify

Módulo 1: Transacciones (Orders, Payments) → CP (PostgreSQL)

Prioridades:

1. Consistency (C) = CRÍTICA

- No duplicar pedidos
- Suma de pagos = total pedido
- Estados de pedido consistentes

2. Partition Tolerance (P) = IMPORTANTE

- Réplicas read-only
- Failover manual

3. Availability (A) = TOLERABLE

- Si primary cae, mejor rechazar writes que duplicar
- Downtime temporal aceptable (< 1 hora)

Decisión: PostgreSQL (CP)

ACID garantizado

Foreign keys

Single point of failure (free tier)

Justificación:

- Un pedido duplicado cuesta dinero y reputación
- Mejor tener 1h de downtime que datos inconsistentes
- Free tier no permite HA real, pero prioriza C sobre A

Módulo 2: Catálogo (Products) → AP (MongoDB)

Prioridades:

1. Availability (A) = CRÍTICA

- Catálogo SIEMPRE accesible
- Ventas perdidas si catálogo no carga

2. Partition Tolerance (P) = IMPORTANTE

- Réplicas en múltiples data centers
- Sharding para escalar

3. Consistency (C) = TOLERABLE

- OK si producto nuevo tarda 1-5 min en aparecer en todos los nodos
- Precio desactualizado por 1 min no es crítico

Decisión: MongoDB (AP)

Alta disponibilidad (3 réplicas)

Eventual consistency OK

Sharding para escalar

Justificación:

- Catálogo no disponible = 0 ventas
- Cliente tolera precio/stock desactualizado 1-2 minutos
- MongoDB replica set: writes en primary, reads en secondaries

Módulo 3: Reviews → AP (MongoDB)

Prioridades:

1. Availability (A) = IMPORTANTE

- Permitir dejar review siempre
- Mostrar reviews existentes siempre

2. Partition Tolerance (P) = IMPORTANTE

- Múltiples replicas

3. Consistency (C) = BAJA

- Review nueva puede tardar 5 min en aparecer = OK
- No es transaccional

Decisión: MongoDB (AP)

Append-only (no conflicts)

Eventual consistency perfecta para este caso

Módulo 4: Analytics → AP (MongoDB)

Prioridades:

1. Availability (A) = CRÍTICA

- Dashboards siempre online
- CEO mirando métricas 24/7

2. Partition Tolerance (P) = BAJA

- Free tier = single node

3. Consistency (C) = MUY BAJA

- Datos de ayer son suficientes
- ETL nocturno actualiza

Decisión: MongoDB (AP)

Pre-agregado (instant queries)

Datos historical (no requieren consistency)

5.3 Trade-offs y Estrategias de Mitigación

Trade-off 1: Consistencia Eventual en Catálogo

Problema:

- MongoDB puede mostrar precio desactualizado durante 1-2 minutos después de cambio

Estrategia de mitigación:

1. Implementar versioning en documentos

```
{  
  "_id": "prod_001",  
  "price": 89.90,  
  "version": 42,  
  "updated_at": ISODate("2026-05-20T14:30:00Z")  
}
```

2. Read Concern "majority" para lecturas críticas (checkout)

```
db.products_catalog.findOne(  
  { _id: "prod_001" },
```

```
{ readConcern: { level: "majority" } } // ← Espera confirmación de mayoría de réplicas  
);
```

3. Write Concern "majority" para escrituras críticas

```
db.products_catalog.updateOne(  
  { _id: "prod_001" },  
  { $set: { price: 79.90, version: 43 } },  
  { writeConcern: { w: "majority", wtimeout: 5000 } } // ← Confirmación en mayoría  
);
```

4. En aplicación: mostrar timestamp de última actualización

"Precio actualizado hace 2 minutos"

Trade-off 2: Sin Alta Disponibilidad en Free Tier

Problema:

- Supabase y MongoDB Atlas M0 no tienen réplicas automáticas
- Downtime si primary falla

Estrategia de mitigación:

1. PostgreSQL (Supabase):

Backups automáticos diarios (7 días retencion)

Scripts pg_dump semanales a Google Drive

Health check cada 5 min (ping desde Uptime Robot)

Failover manual (restaurar desde backup)

2. MongoDB (Atlas M0):

Réplicas read-only (3 nodos en free tier)

Auto-failover SI disponible en M0

Backups manuales con mongodump

3. Arquitectura de resiliencia:

[App] → [PostgreSQL Primary] ← Ping cada 5 min

↓ (si falla)

↓

[Modo degradado]

↓

[MongoDB cache] → Mostrar datos del cache (5 min desactualizados)

↓

[Mensaje al usuario: "Experiencia limitada, trabajando en restauración"]

Trade-off 3: Duplicación de Datos (Products)

Problema:

- Products existe en PostgreSQL (master) y MongoDB (cache)

- ¿Cómo mantener sincronizados?

Estrategia de mitigación:

ETL cada 1 hora - sincronizar products

def sync_products_to_mongodb():

 # 1. Extraer de PostgreSQL (source of truth)

 pg_products = pg_cursor.execute("""

```

SELECT
    p.product_id,
    p.product_category_name,
    c.product_category_name_english,
    p.product_weight_g,
    p.product_length_cm,
    p.product_height_cm,
    p.product_width_cm,
    p.product_photos_qty
FROM products p
LEFT JOIN product_category_name_translation c
    ON p.product_category_name = c.product_category_name
WHERE p.updated_at >= NOW() - INTERVAL '1 hour' -- Solo cambios recientes
""").fetchall()

```

2. Transformar y cargar en MongoDB

```

for row in pg_products:
    mongo_doc = {
        "_id": row['product_id'],
        "category": {
            "name_pt": row['product_category_name'],
            "name_en": row['product_category_name_english']
        },
        "dimensions": {
            "weight_g": row['product_weight_g'],
            "length_cm": row['product_length_cm'],

```

```

        "height_cm": row['product_height_cm'],
        "width_cm": row['product_width_cm']
    },
    "photos_qty": row['product_photos_qty'],
    "synced_at": datetime.utcnow(),
    "version": row['updated_at'].timestamp()
}

```

```

mongo_db.products_catalog.update_one(
    {"_id": mongo_doc["_id"]},
    {"$set": mongo_doc},
    upsert=True
)

```

```

print(f'Synced {len(pg_products)} products to MongoDB')

```

Validación de consistencia (semanal)

```

def validate_sync_consistency():

```

```

    pg_count = pg_cursor.execute("SELECT COUNT(*) FROM products").fetchone()[0]

```

```

    mongo_count = mongo_db.products_catalog.count_documents({})

```

```

    if pg_count != mongo_count:

```

```

        alert("Product count mismatch: PG={pg_count}, Mongo={mongo_count}")

```

Trade-off 4: Sin Sharding en Free Tier

Problema:

- MongoDB Atlas M0 no soporta sharding
- ¿Cómo escalar a 10M+ documentos?

Estrategia de mitigación:

1. INMEDIATO (Free Tier):

Índices optimizados (cobertura de queries frecuentes)

Aggregation pipeline eficiente

Eliminar documentos antiguos (> 3 años) manualmente

2. A FUTURO (Upgrade a M10+):

Shard key: { "customer.state": 1, "timestamps.purchase": 1 }

Distribuye datos por región geográfica

Queries filtradas por estado usan solo 1 shard

3. AHORA (Preparación):

Diseñar shard key en schema actual

Documentar estrategia de sharding

Queries siempre incluyen shard key cuando sea posible

// Ejemplo de query optimizada para futuro sharding

```
db.order_history.find({  
  "customer.state": "SP", // ← Parte del shard key  
  "timestamps.purchase": { $gte: ISODate("2026-01-01") } // ← Parte del shard key  
}).limit(100);
```

5.4 Flujos de Sincronización

Flujo 1: Creación de Pedido (Write-Heavy)

1. Usuario hace checkout → API recibe request

2. API → PostgreSQL (ACID transaction):

```
BEGIN;
```

```
INSERT INTO orders (...);
```

```
INSERT INTO order_items (...); -- 3 items
```

```
INSERT INTO order_payments (...); -- 2 payments
```

```
COMMIT; -- ← Si falla cualquier INSERT, todo se revierte
```

3. PostgreSQL → Respuesta a API:

```
"Order created: order_xyz789"
```

4. API → Responde a usuario:

```
"Pedido confirmado #XYZ789"
```

5. (ASYNC) Background job cada 5 min:

```
ETL → Leer orders nuevos de PostgreSQL
```

```
→ Transformar a documento MongoDB (denormalizado)
```

```
→ Insertar en order_history collection
```

6. Cliente consulta historial 3 min después:

Puede NO aparecer aún (lag de 5 min)

→ Fallback: si no existe en MongoDB, leer de PostgreSQL

Código de sincronización:

Cron job cada 5 minutos

def sync_orders_to_mongodb():

Obtener orders modificados en los últimos 5 min

new_orders = pg_cursor.execute("""

SELECT

o.order_id,

o.order_status,

o.order_purchase_timestamp,

json_agg(DISTINCT jsonb_build_object(

'item_id', oi.order_item_id,

'product_id', oi.product_id,

'price', oi.price

)) AS items,

json_agg(DISTINCT jsonb_build_object(

'sequential', op.payment_sequential,

'type', op.payment_type,

'value', op.payment_value

)) AS payments

FROM orders o

LEFT JOIN order_items oi ON o.order_id = oi.order_id

LEFT JOIN order_payments op ON o.order_id = op.order_id

WHERE o.updated_at >= NOW() - INTERVAL '5 minutes'

GROUP BY o.order_id, o.order_status, o.order_purchase_timestamp

""").fetchall()

for order in new_orders:

```
    mongo_doc = {  
        "_id": order['order_id'],  
        "status": order['order_status'],  
        "timestamps": {"purchase": order['order_purchase_timestamp']},  
        "items": json.loads(order['items']),  
        "payments": json.loads(order['payments']),  
        "synced_at": datetime.utcnow()  
    }  
  
    mongo_db.order_history.update_one(  
        {"_id": mongo_doc["_id"]},  
        {"$set": mongo_doc},  
        upsert=True  
    )
```

Flujo 2: Búsqueda en Catálogo (Read-Heavy)

1. Usuario busca "mouse wireless"
2. API → MongoDB (products_catalog):

```
db.products_catalog.find(  
    $text: { $search: "mouse wireless" }  
}).limit(20);
```

3. MongoDB → Responde en ~20ms:

```
[{ _id: "prod_001", name: "Mouse Logitech MX Master 3", ... }, ...]
```

4. API → Renderiza resultados al usuario

5. NO hay sincronización en tiempo real:

- Products se actualizan SOLO cuando seller cambia precio/stock
- Lag de 1 hora es aceptable (ETL cada 1h)

Flujo 3: Dashboard de Analytics (Batch ETL)

1. ETL nocturno (2:00 AM):

a. PostgreSQL → Extraer ventas del día anterior:

```
SELECT
    DATE(order_purchase_timestamp) AS date,
    product_category_name,
    COUNT(*) AS total_orders,
    SUM(price) AS revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE order_purchase_timestamp::date = CURRENT_DATE - INTERVAL '1 day'
    AND order_status = 'delivered'
GROUP BY DATE(order_purchase_timestamp), product_category_name;
```

b. Transformar → Documento agregado

c. MongoDB → Insertar en daily_sales_analytics:

```
db.daily_sales_analytics.insertOne({
  "_id": "2026-05-20_electronics",
  "date": ISODate("2026-05-20"),
  "category": "electronics",
  "metrics": { "total_orders": 1247, "revenue": 156789.50 }
});
```

2. Dashboard consulta datos:

- MongoDB (instant query <10ms)
- Muestra datos de ayer (lag de 24h es aceptable)

6. Anexos

6.1 Diccionario de datos completo

Tabla	Columns	PK	FKs	Indexes	Tamaño Estimado
customers	6	1	0	4	15 MB
orders	8	1	1	5	25 MB
order_items	7	2	3	4	30 MB
order_payments	5	2	1	3	20 MB
sellers	5	1	0	4	1 MB
products	10	1	1	3	10 MB
product_category_name_translation	2	1	0	2	0.01 MB
geolocation	5	3	0	2	150 MB 

6.2 Scripts SQL Preliminares

En repositorio ver carpeta en: `postgresql/schema/`

Archivos:

- `01\_create\_types.sql` (ENUMs)
- `02\_create\_tables.sql` (Todas las tablas)
- `03\_create\_indexes.sql` (Índices de performance)
- `04\_create\_constraints.sql` (Foreign keys, checks)

- `05\_create\_functions.sql` (Stored procedures)
- `06\_create\_triggers.sql` (Triggers de validación)
- `07\_create\_views.sql` (Vistas materializadas)

Referencias

1. PostgreSQL Documentation. (2025). PostgreSQL 16 Official Documentation.
 2. MongoDB Inc. (2025). MongoDB Manual - Data Modeling.
 3. Brewer, E. (2000). "Towards Robust Distributed Systems" - CAP Theorem.
 4. Elmasri & Navathe. (2016). "Fundamentals of Database Systems" (7th ed.).
 5. Kleppmann, M. (2017). "Designing Data-Intensive Applications". O'Reilly.
 6. Olist Dataset. (2018). Brazilian E-Commerce Public Dataset. Kaggle.
- MongoDB, Inc. (2025). Data modeling introduction. MongoDB Manual.